

SMARTFLOW BEAUTY

# План рефакторинга

Для подготовки версии 1.0 и первых пилотов

MVP • Пилотное тестирование • 2026

Версия документа: 1.0

Состояние проекта: MVP / подготовка к пилотному тестированию

Главный принцип: безопасность и сохранение работающего поведения важнее архитектурной «чистоты».

## 1. Цель следующего этапа

Подготовить SmartFlow Beauty к пилотам в реальных салонах без бесконечного рефакторинга. Изменения следует делать только тогда, когда они снижают риск, упрощают сопровождение, устраняют подтверждённое дублирование или нужны для новой функции.

На текущем этапе уже выполнены крупные структурные работы: код разделён по областям ответственности, удалены устаревшие Owner/финансовые концепции и CalendarCache, сущности хранят собственные названия, конфигурация ограничена allowlist, а интерфейс локализован.

## 2. Правила выполнения

1. Один чекпоинт должен иметь понятную эксплуатационную ценность.
2. Механические переносы допускается объединять, но изменение поведения требует отдельной регрессии.
3. До удаления функции необходимо доказать, что она не вызывается роутерами, триггерами, миграциями или диагностикой.
4. Любое изменение структуры Google Sheets сопровождается повторно-безопасной миграцией.
5. Документация и статический валидатор обновляются в том же чекпоинте.
6. Токены, Telegram ID клиентов и данные салона не попадают в Git, логи и тестовые артефакты.
7. Не менять поля и бизнес-правила «на будущее», если их назначение ещё не согласовано.

## 3. Приоритет P0: до первого внешнего пилота

### 3.1 Секреты и безопасное развёртывание

Рабочие токены удалены из копируемого шаблона и текущей схемы Settings. Production использует только Script Properties `SMARTFLOW_CLIENT_BOT_TOKEN` и `SMARTFLOW_ADMIN_BOT_TOKEN`; legacy-установки последовательно запускают миграцию переноса и удаления старых строк.

Критерии готовности:

- новый шаблон не содержит действующих токенов и персональных Telegram ID;
- токены никогда не выводятся в AuditLog и сообщения бота;
- есть repeat-safe миграция и процедура ротации токена;
- техническая инструкция проверена на чистой копии таблицы.

Реализовано в коде: runtime читает секреты только из Script Properties, health check блокирует release при отсутствии любого токена, а процедуры миграции, удаления legacy-строк и ротации описаны в deployment guide. Проверка чистой установки остаётся частью внешней регрессии чекпоинта.

## 3.2 Автоматизированные smoke-тесты критических потоков

Текущий статический валидатор хорошо ловит дубли, отсутствующие ключи и возврат удалённых концепций, но не проверяет бизнес-поведение. Следующий разумный шаг — тестируемые чистые функции и небольшой набор сценарных тестов с адаптерами вместо Telegram, Sheets и Calendar.

Минимальный набор:

- маршрутизация `book_appointment` и финальное создание заявки;
- подтверждение и отклонение администратором без дублей;
- перенос записи с очисткой напоминания и подтверждения;
- отмена записи и удаление связанного Calendar-события;
- разные мастера могут быть заняты параллельно;
- ручное Calendar-событие занимает слот только своего мастера;
- проверка прав Admin Bot;
- повторный webhook update не обрабатывается дважды;
- синхронизация истории визитов не создаёт дубль.

Защита от повторного webhook update уже покрыта детерминированными тестами: точный повтор выполняется один раз, неуспешный обработчик не помечается завершённым, порядок ID не приводит к потере, а истории двух ботов изолированы. Остальные сценарные тесты этого раздела остаются обязательными.

## 3.3 Проверка входных данных и отказоустойчивость интеграций

Централизовать безопасную обработку ответов Telegram API и Calendar API: проверять HTTP-код, `ok`, отсутствие календаря, удалённое событие и недостаточные права. Пользователю показывать нейтральное сообщение, а техническую причину записывать в AuditLog без секретов.

Отдельно проверить все команды изменения данных: операция должна завершаться полностью либо оставлять состояние, пригодное для безопасного повтора.

## 3.4 Release checklist и версия

Добавить единую версию приложения, журнал изменений и короткий повторяемый release checklist: validate, regression, `clasp push`, deployment, migrations, triggers, webhooks, health check и rollback decision.

Выполнено для пилотного чекпоинта: версия хранится в `VERSION` и `AppsScript/Version.gs`, health check возвращает её в отчёте, изменения ведутся в `CHANGELOG.md`, а обязательные шаги выпуска собраны в `Docs/Guides/Version 1.0/RELEASE_CHECKLIST.md`.

# 4. Приоритет P1: во время первых пилотов

## 4.1 Упрощение крупных роутеров по фактическим проблемам

Не переписывать маршрутизацию целиком. Сначала измерить, какие файлы чаще дают регрессии. Затем вынести таблицы соответствий «команда → обработчик» и «состояние → обработчик» там, где это уменьшает длинные условные цепочки без изменения приоритетов команд.

Особое правило: глобальная область Apps Script сохраняется, поэтому имена функций должны оставаться уникальными и проверяться валидатором.

## 4.2 Единый результат команд записи

Команды Sheets сейчас возвращают разные типы результатов. Полезно постепенно стандартизировать результат мутации: `ok`, идентификатор, код ошибки и безопасное сообщение. Это упростит обработку частичных отказов Calendar/Telegram.

## 4.3 Производительность live Calendar-запросов

CalendarCache возвращать не следует. Вместо этого во время пилота измерять длительность конкретных экранов и число Calendar-вызовов. Оптимизировать только подтверждённые узкие места: группировать запросы по уникальному календарю, читать Sheets пакетно, не выполнять одинаковое обогащение сущностей в цикле.

Допустим только короткоживущий runtime-cache справочников. Он не должен становиться источником истины для записей.

## 4.4 Диагностика и health check

Добавлена безопасная read-only функция `runSmartFlowHealthCheck()`, которая возвращает:

- наличие обязательных листов и колонок;
- корректность TimeZone и URL deployment;
- доступность календарей мастеров;
- наличие двух webhook;
- наличие триггеров напоминаний и истории визитов;
- отсутствие дублирующихся installable triggers;
- наличие хотя бы одного администратора и получателя заявок.

Не включать токены и персональные данные в результат.

## 4.5 Архивация и рост таблиц

Во время пилота измерить рост `AuditLog`, `UserSessions`, `UserStates`, `Requests`, `Appointments` и истории. Только после этого определить retention policy и repeat-safe архивирование. Не удалять бизнес-историю без письменного правила владельца салона.

# 5. Приоритет P2: после подтверждения пилотами

## 5.1 Установщик новой копии

Сделать одну административную функцию начальной установки: проверка схемы, добавление недостающих Messages, установка безопасных значений по умолчанию, создание нужных триггеров и итоговый отчёт. Она не должна перезаписывать заполненные значения.

## 5.2 Более строгая модель доступа

Рассмотреть роли «владелец конфигурации» и «оператор записей» только если салоны подтвердят потребность. Сейчас вводить новые роли преждевременно. До этого достаточно AdminTelegramIds и защищённого доступа к самой Google-таблице.

## 5.3 Наблюдаемость без тотального логирования

Не возвращать глобальный `EnableLogs`. Вместо него использовать структурированные события только для важных операций и ошибок, correlation ID для одной заявки и ограниченные диагностические функции, запускаемые техническим специалистом.

## 5.4 UX по результатам использования

Собирать реальные затруднения пользователей: непонятные кнопки, лишние шаги, возвраты «Назад», длинные списки и ошибки ввода. Менять UX на основании наблюдений, а не теоретического упрощения.

## 6. Что сознательно не делать сейчас

- не вводить чат клиента с администратором до версии после 1.0;
- не добавлять массовые рассылки без согласия пользователей и политики Telegram;
- не возвращать цены и валюту;
- не создавать новый CalendarBusyBlock;
- не локализовать названия мастеров, услуг и локаций через Messages;
- не создавать микросервисы или отдельную базу данных для раннего пилота;
- не переписывать весь проект на классы или TypeScript только ради стиля;
- не менять структуру полей, назначение которых пока не определено.

## 7. Рекомендуемые чекпоинты

### Чекпоинт А — безопасность установки

Секреты, чистый шаблон, health check, release checklist и проверенная инструкция развёртывания. После него — полная регрессия и пилотная установка в отдельном тестовом салоне.

### Чекпоинт В — тестовый контур

Адаптеры внешних интеграций и smoke-тесты критических потоков. После него регрессия сценариев создания, подтверждения, переноса, отмены, напоминания и параллельной работы мастеров.

### Чекпоинт С — устойчивость интеграций

Единая обработка Telegram/Calendar ошибок и безопасный повтор операций. После него тесты с неверным Calendar ID, удалённым событием, отозванными правами и временной ошибкой Telegram.

### Чекпоинт D — решения по данным пилота

Только измеренные оптимизации, retention policy и UX-исправления. После него можно переключаться на новую функциональность без продолжения общего рефакторинга.

## 8. Критерий завершения рефакторинга

Рефакторинг считается достаточным для версии 1.0, когда чистая установка воспроизводима по документации, секреты защищены, критические сценарии покрыты smoke-тестами, health check обнаруживает типовые ошибки, а два последовательных пилота проходят без дефектов потери/дублирования записей. После этого каждое новое изменение должно быть связано с конкретной функцией, дефектом, безопасностью или измеренной производительностью.